

IntelliSense Support for Discord-Bot Development with Discord.Net

Merwa Stefan



BACHELORARBEIT (EXPOSÉ)

eingereicht am
Fachhochschul-Bachelorstudiengang

Software Engineering

in Hagenberg

im Juni 2023

Contents

Abstract	ii
Exposé	1
1 Introduction	1
1.1 What is Discord?	1
1.2 What are Discord bots?	1
1.3 What is Discord.Net?	2
1.4 What is a Visual Studio Extension?	2
2 State of the Art	3
2.1 Discord.Net	3
2.2 Roslyn	3
2.3 Editor API	3
3 Methodology	3
4 Expected Results	4
References	5
Literature	5
Online sources	5

Abstract

Developing third party applications for Discord can require repeated coding inputs. Something like a code base for specific needs could be generated or entity ids that can be copied from the Discord App could be provided directly while coding in the IDE.

Exposé

1 Introduction

Developing Discord bots can be exhausting and time consuming, but there are several ways to make it more comfortable. The following scenario will handle the creation of an extension for Visual Studio. This extension prototype will ease up the coding process for Discord bots, by adapting the IntelliSense process for unique identifiers of discord entities.

1.1 What is Discord?

“Discord is a voice, video, and text chat app that’s used by tens of millions of people ages 13+ to talk and hang out with their communities and friends” [3].

Discord has its own vocabulary, for a better understanding some terms are explained. Servers are the spaces on Discord. They are made by specific communities and friend groups. The vast majority of servers are small and invitation-only. Some larger servers are public. Any user can start a new server for free and invite their friends to it. Discord servers are organized into text and voice channels, which are usually dedicated to specific topics and can have different rules. [3]

It is important to note, that each of these entities in Discord have a unique identifier, that can be used to access the identified entity through the Discord-API.

1.2 What are Discord bots?

A Discord bot is a program or script designed to run on Discord that can perform a variety of functions and automate tasks within Discord servers. These functions mainly contain of

1. Moderation: Bots can help manage and moderate Discord servers by enforcing rules, detecting and handling inappropriate content, and issuing warnings or punishments to users who violate server guidelines.
2. Automation: Bots can automate repetitive tasks, such as welcoming new members, assigning roles based on certain criteria, and cleaning up messages or channels.
3. Information Retrieval: Bots can fetch information from external sources or APIs and provide it within the Discord server. This can include weather updates, news,

or data from specific websites.

4. Fun and Games: Many Discord bots are created for entertainment purposes. They can offer games, quizzes, or other interactive elements to keep server members engaged.
5. Music: Some bots are designed specifically for playing music in voice channels. They can queue up songs, adjust volume, and provide a music listening experience for server members.
6. Custom Commands: Server owners can create custom commands that trigger specific actions or responses from the bot. This allows for a high degree of customization based on the server's needs.
7. Integration with External Services: Bots can integrate with external services and platforms, allowing users to interact with those services directly from Discord. For example, bots can interact with Google Calendar, GitHub, or other APIs. [1]

1.3 What is Discord.Net?

Discord.Net is an asynchronous and multi-platform .NET Library used to interface with the Discord API. It provides a set of tools and abstractions to make this task easier for developers. The key features include:

- `DiscordSocketClient`: This is a class that represents the connection to the Discord gateway and provides various events.
- `CommandService`: Discord.Net includes a command framework that simplifies the process of creating and handling commands in a bot.
- `WebSocket` and `REST API Handling`: Discord.Net handles the communication with the Discord API not only through `WebSocket` connections, but also through `REST API` calls that don't require real time communication.
- `Entities`: Discord.Net represents Discord objects (users, channels, guilds, messages etc.) as classes. [4]

1.4 What is a Visual Studio Extension?

Extensions serve as add-ons enabling you to personalize and enrich your Visual Studio experience by introducing new features or integrating existing tools. Extensions vary in complexity but share a common goal: to boost productivity and align with your workflow. In theory, every part of Visual Studio can be extended, practice shows what most people like to extend are commands, menus and toolbars, windows, IntelliSense and projects. There are also many project templates provided for these scenarios. [8]

Extension task

The Visual Studio extension prototype is capable of analysing the programmers code on the fly and provide instant IntelliSense support for unique identifiers of Discord entities, for Discord.Net (This can be extended for other .NET wrappers for Discord development). The data, that is provided for the IntelliSense interaction, can also be generated dynamically inside Visual Studio.

2 State of the Art

2.1 Discord.Net

To generate the data dynamically inside Visual Studio, Discord's API needs to be accessed. The Discord.Net wrapper will not only be the target of the extension, but is also used to build a connection to the API and retrieve the required data. There are many other wrappers for Discord for many different languages like D++ for *C++* or discord.py for *Python*. There is also another wrapper for .NET called DSharpPlus, but its way less popular and not very well documented, so the preferred choice for .NET is Discord.Net.

2.2 Roslyn

The .NET Compiler Platform aka. MS Roslyn is used for C# and VB to enable building tools and extensions to perform routine programming tasks like source code compilation, code analysis, code refactoring and code generation. Roslyn is divided into multiple programming layers that expose public APIs to write customized tools. These APIs contain of 4 layers:

- Code Analysis: Core syntax and semantic representation.
- Workspaces: Set of logically related source files in projects and solutions.
- Features: IDE features build on top of the Code Analysis and Workspace API like IntelliSense.
- Visual Studio: Visual Studio workspace and project system. [2]

2.3 Editor API

To interact with the IntelliSense API, Visual Studio provides an Editor API [9] that comes with various functionalities. Roslyn also provides functionality to interact with completions, but it's not as technically mature as the Editor API. Besides these scenario, where the implementation is language and IDE specific, there exists a protocol for standardizing messages, so language specific domains are not necessary, called Language Server Protocol. Software development tools communicate with the language server using this protocol over JSON-RPC. [5] This would usually be the preferred choice to implement language extensions, but in this case scenario, the language support is specifically for Discord.Net, so the Editor API provides more than enough functionality.

3 Methodology

To fulfill the requirements for this Visual Studio Extension, several utilities, provided by Microsoft, are necessary. To analyze code, the VS Solution needs to be loaded. That can be achieved either for testing purposes via MSBuild, or via the ComponentModel service provided by VisualStudio, that contains the VisualStudioWorkspace with all items like projects and documents. The containing code inside the projects can then be processed via MS Roslyn. "The .NET Compiler Platform SDK provides the tools you

need to create custom diagnostics (analyzers), code fixes, code refactoring, and diagnostic suppressors that target C# or Visual Basic code.” [7]

To provide custom IntelliSense items, so called completion items, the Editor API is used. “Completion in VS uses MEF to discover extensions. Specifically, VS is looking for exports of type *IAsyncCompletionSourceProvider*, *IAsyncCompletionItemManagerProvider*, *IAsyncCompletionCommitManagerProvider* and *ICompletionPresenterProvider*.” [9]

MEF stands for Managed Extensibility Framework, which is a library for the development of lightweight and extensible applications. It allows developers to discover and use extensions with no configuration required and lets them easily encapsulate code to avoid fragile hard dependencies. MEF also allows applications to be reused not only inside, but across different applications. It is an integral part of the .NET Framework 4 and available wherever the .NET Framework is used, whether its for WPF, ASP.NET or any other application.[6]

4 Expected Results

The Visual Studio Extension prototype will provide 3 commands for configuring data generation, generating data from Discord and loading the data per project. It will also analyze the developers code and provides the context specific data to the completion API, which leads to being suggested as IntelliSense item. These items will consist of static discord entity ids.

References

Literature

- [1] Advait Agnihotri, Harshit Gautam, and Hari Singh. “Discord Bot” (2022) (cit. on p. 2).
- [2] Manish Vasani. *Roslyn Cookbook*. Packt Publishing Ltd, 2017 (cit. on p. 3).

Online sources

- [3] Discord. *What is Discord?* URL: <https://discord.com/safety/360044149331-what-is-discord> (visited on 07/08/2023) (cit. on p. 1).
- [4] Discord.Net. *Discord.Net*. URL: <https://discordnet.dev/> (visited on 11/23/2023) (cit. on p. 2).
- [5] Microsoft. *Language Server Protocol*. URL: <https://learn.microsoft.com/de-de/visual-studio/extensibility/language-server-protocol?view=vs-2022> (visited on 09/10/2023) (cit. on p. 3).
- [6] Microsoft. *Managed Extensibility Framework*. URL: <https://learn.microsoft.com/en-us/dotnet/framework/mef/> (visited on 09/10/2023) (cit. on p. 4).
- [7] Microsoft. *Tutorial: Write your first analyzer and code fix*. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/tutorials/how-to-write-csharp-analyzer-code-fix> (visited on 09/10/2023) (cit. on p. 4).
- [8] Microsoft. *Visual Studio Extension*. URL: <https://learn.microsoft.com/en-us/visualstudio/extensibility/starting-to-develop-visual-studio-extensions?view=vs-2022> (visited on 11/23/2023) (cit. on p. 2).
- [9] Amadeusz Wiczorek. *Async Completion API*. URL: <https://github.com/microsoft/vs-editor-api/issues/9> (visited on 09/10/2023) (cit. on pp. 3, 4).